DEVOPS

LAB 04

Simple Python Database Connections

MySQL and MongoDB with Python, Docker and Ubuntu

Student Name	Eman Fatima
Registration Number	2023-BCS-025
Section	A
GitHub Username	Eman-Fatima-7125
Ubuntu Identity	eman-fatima-7125@ubuntu
Lab Number	04
Date	26 October 2026
Databases Fork	https://github.com/Eman-Fatima-7125/Databases
Submission Directory	DevOps/Labs/Lab04

Table of Contents

1. Introduction and Objectives 2

2. Getting Started: Evidence Directory 3

3. Task 1: Fork and Clone the Databases Repository 3

4. Task 2: Install Docker, Docker Compose and Python 5

5. Task 3: Start the Database Services 7

6. Task 4: MySQL with Values in the Code 9

7. Task 5: MongoDB with Values in the Code 10

8. Task 6: MongoDB with os.getenv() 11

9. Task 7: MongoDB with a Credentials File 13

10. Exam Evaluation Questions 16

11. Overall Summary and Conclusion 17

12. Submission Details 18

1. Introduction and Objectives

Modern applications rarely store their own data; they connect to a separate database server by presenting a username, a password, an endpoint (host and port) and a database name. This laboratory demonstrates that idea in four short stages, all implemented in one Python file, *db_connect.py*. Python runs directly on an Ubuntu server, while Docker is used only to run the MySQL and MongoDB database services. Each stage connects, sends a real request and prints which database answered, so the target can be verified from the database itself.

Learning objectives

- Explain the username, password, endpoint and database name used for a connection.
- Connect to MySQL and MongoDB using a short Python program.
- Explain the settings embedded in a MongoDB URI.
- Use *os.getenv()* to run unchanged Python code against dev, test and simulated prod.
- Use *configparser* to read the same settings from an INI credentials file.
- Read the actual server's stored environment value to confirm the target.
- Capture readable evidence of every database connection.

Environment used

Component	Details
Operating system	Ubuntu 26.04.1 LTS (x86_64), reached over SSH
Server hostname / user	ubuntu / eman-fatima-7125
Git	version 2.53.0
Docker Engine	29.1.3 (docker.io package)
Python	3.14.4 inside a virtual environment (.venv)
Python packages	pymongo 4.19.0, mysql-connector-python 9.7.0, dnspython 2.9.0
Databases	MySQL 8.4 and MongoDB 7.0 (three instances) in Docker containers

The four stages

Stage	Database	Source of connection settings
1 (Task 4)	MySQL	Written directly in the code
2 (Task 5)	MongoDB	Written directly in the code
3 (Task 6)	MongoDB	Environment variables via os.getenv()
4 (Task 7)	MongoDB	Credentials file (credentials.ini) via configparser

Security note. All credentials in this lab are public dummy values for disposable classroom databases. In accordance with the lab rules, passwords are masked as ******** in the code listings of this document.

2. Getting Started: Evidence Directory

Before any technical work, the evidence directory was created inside the existing *DevOps* repository. The command *mkdir -p Labs/Lab04/screenshots* creates the full path in one step and does not fail if parts of it already exist. This directory holds the README, this solution document and every screenshot, whereas the programming work is done in the separate *Databases* fork.

```
cd ~/DevOps
mkdir -p Labs/Lab04/screenshots
```



Figure 1: Creating the Labs/Lab04/screenshots evidence directory in the DevOps repository.

3. Task 1: Fork and Clone the Databases Repository

A *fork* is a personal copy of someone else's repository stored under your own GitHub account. Forking the instructor's *Databases* starter repository gives a safe, editable copy while keeping a link to the original. The fork page shows *Eman-Fatima-7125/Databases* with the note "forked from WaqasSaleem97/Databases" and the message that the branch is up to date with the original *main* branch, confirming that the fork was created correctly.

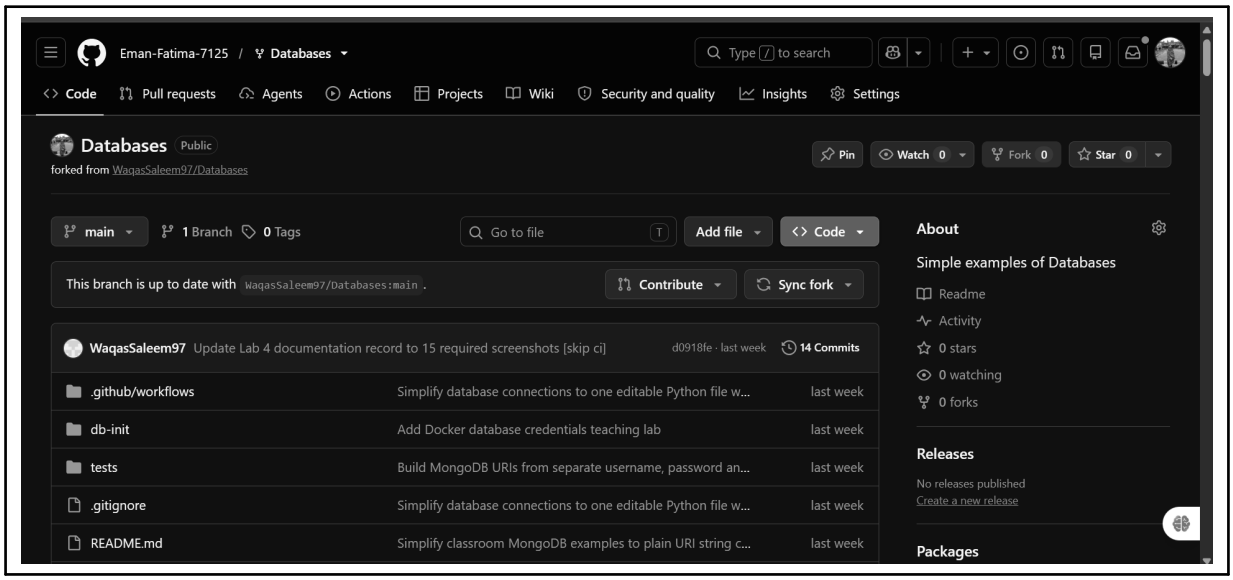


Figure 2: GitHub page of the fork Eman-Fatima-7125/Databases, forked from WaqasSaleem97/Databases.

The fork was then cloned onto the Ubuntu server with *git clone*, which downloads the full repository history and working files to the local machine. The output reports that all 101 objects were received and all 30 deltas were resolved, which indicates a complete and successful clone.

```
git clone https://github.com/eman-fatima-7125/Databases.git
```

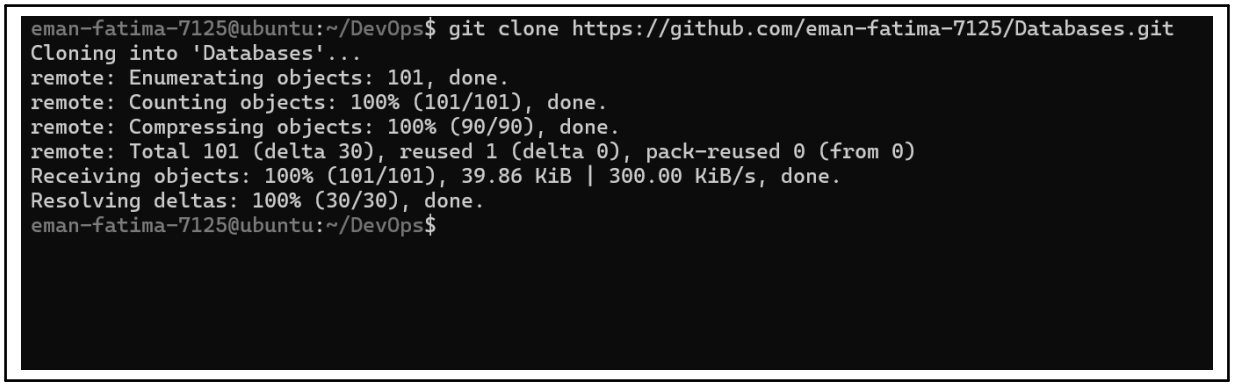


Figure 3: Cloning the Databases fork onto the Ubuntu server.

Summary: The fork contains the code that is edited during the lab, while the DevOps repository stores the evidence.

4. Task 2: Install Docker, Docker Compose and Python

This task prepares the server. Docker Engine (package *docker.io*) runs containers, and the Docker Compose plug-in starts several related containers from one *compose.yaml* file. The user was added to the *docker* group so that Docker commands work without *sudo*; this change only takes effect after a full sign-out and a new SSH session. The commands used were:

```
sudo apt update -y
sudo apt install -y docker.io curl
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
    -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sudo systemctl start docker
sudo usermod -aG docker $USER
exit
```

After reconnecting over SSH, the installations were verified. The screenshot shows the SSH login to the Ubuntu server, followed by *git --version* (2.53.0) and *docker --version* (29.1.3), together with the *docker compose version* command.

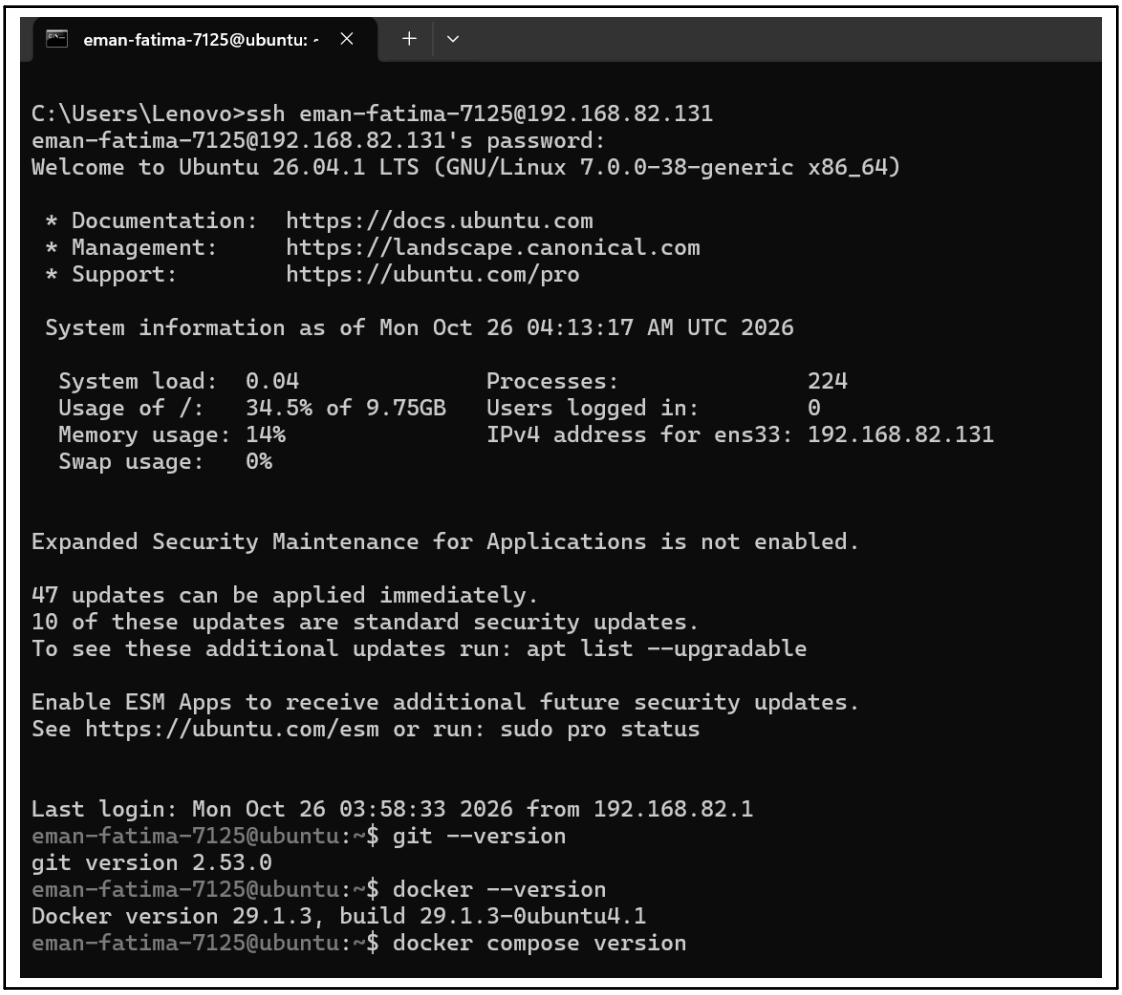


Figure 4: SSH login and verification of the Git, Docker and Docker Compose installations.

Python and virtual environment

Python 3, *venv* and *pip* were installed with apt (they were already at the newest version). A virtual environment named *.venv* was created inside *~/Databases* so that the project's libraries stay isolated from the system Python. After activation, *pip* installed the three required packages listed in *requirements.txt*. The prompt prefix *(.venv)* confirms that the environment is active.

```
sudo apt install -y python3 python3-venv python3-pip
cd ~/Databases
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python --version
```

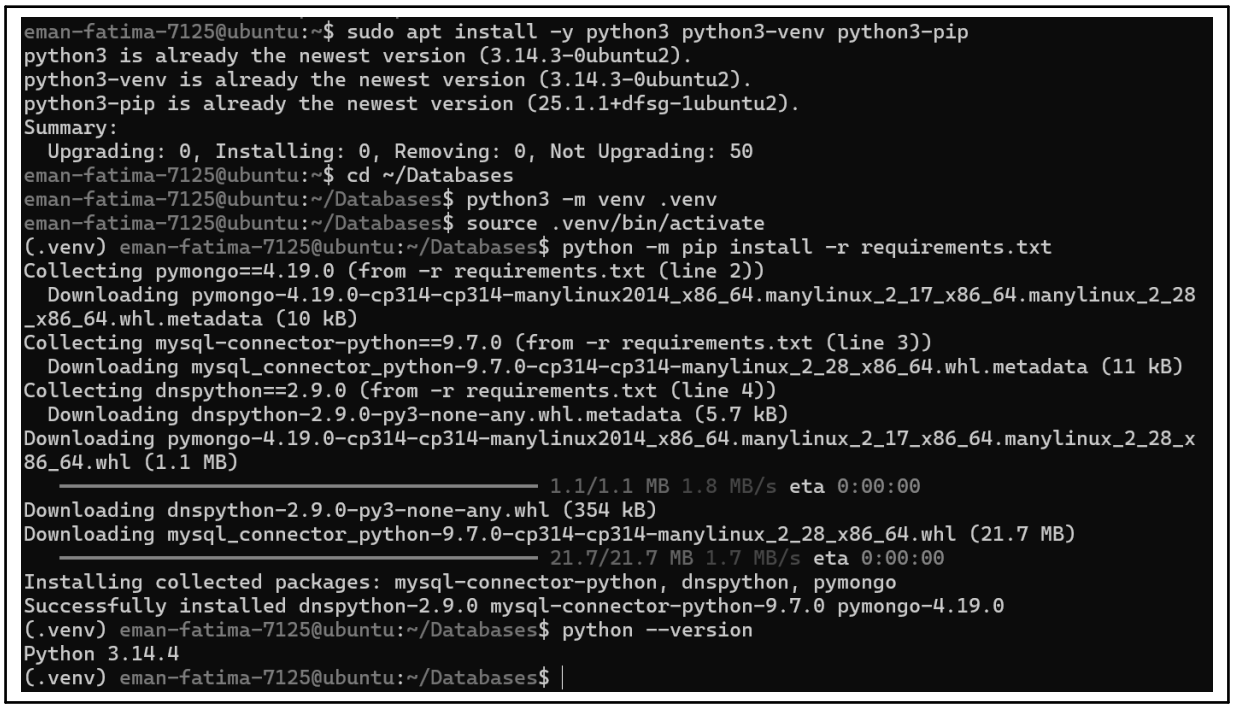


Figure 5: Installing Python, creating the virtual environment and installing the dependencies.

Summary: The server now has Git, Docker, Docker Compose and a Python environment with the PyMongo and MySQL connector libraries.

5. Task 3: Start the Database Services

The supplied *compose.yaml* defines four database services: *mysql*, *mongo-dev*, *mongo-test* and *mongo-prod*. The command *docker compose up -d* starts them in the background (detached mode), and *docker compose ps* lists their state. The screenshot shows that the MongoDB 7.0 image was pulled, that all four containers were started or already running, and that each service is mapped to a port on the local loopback address 127.0.0.1. At the moment of capture the MySQL container reported *healthy*, and the three MongoDB containers were up and in their initial health-check phase.

```
docker compose up -d --wait mysql mongo-dev mongo-test mongo-prod
docker compose ps
```

```
eman-fatima-7125@ubuntu:~/Databases$ docker compose pull mongo-dev mongo-test mongo-prod
[+] pull 11/11
 ✔ Image mongo:7.0 Pulled                                                    189.9s
eman-fatima-7125@ubuntu:~/Databases$ docker compose up -d
[+] up 4/4
 ✔ Container databases-mysql-1       Running                                   0.0s
 ✔ Container databases-mongo-dev-1   Started                                   6.2s
 ✔ Container databases-mongo-prod-1  Started                                   4.2s
 ✔ Container databases-mongo-test-1  Started                                   3.2s
eman-fatima-7125@ubuntu:~/Databases$ docker compose ps
NAME                    IMAGE       COMMAND                  SERVICE      CREATED          STATUS                             PORTS
databases-mongo-dev-1   mongo:7.0   "docker-entrypoint.s…"   mongo-dev    15 seconds ago   Up 11 seconds (health: starting)   127.0.
0.1:27017->27017/tcp
databases-mongo-prod-1  mongo:7.0   "docker-entrypoint.s…"   mongo-prod   15 seconds ago   Up 12 seconds (health: starting)   127.0.
0.1:27019->27017/tcp
databases-mongo-test-1  mongo:7.0   "docker-entrypoint.s…"   mongo-test   15 seconds ago   Up 13 seconds (health: starting)   127.0.
0.1:27018->27017/tcp
databases-mysql-1       mysql:8.4   "docker-entrypoint.s…"   mysql        38 minutes ago   Up 38 minutes (healthy)            33060/
tcp, 127.0.0.1:3307->3306/tcp
```

Figure 6: Starting the four database containers and listing their status.

Next, *docker compose port* was used to display the host endpoint that Python must use for each service. Each MongoDB container listens on its internal port 27017, but Docker publishes them on different host ports so that all three can run side by side. The resulting mapping is summarized below.

```
eman-fatima-7125@ubuntu:~/Databases$ docker compose port mysql 3306
127.0.0.1:3307
eman-fatima-7125@ubuntu:~/Databases$ docker compose port mongo-dev 27017
127.0.0.1:27017
eman-fatima-7125@ubuntu:~/Databases$ docker compose port mongo-test 27017
127.0.0.1:27018
eman-fatima-7125@ubuntu:~/Databases$ docker compose port mongo-prod 27017
127.0.0.1:27019
eman-fatima-7125@ubuntu:~/Databases$
```

Figure 7: Host ports published for the MySQL, MongoDB dev, test and prod services.

Database	Host endpoint	Database name	Username
MySQL	127.0.0.1:3307	school_mysql	mysql_student
MongoDB dev	127.0.0.1:27017	school_dev	dev_user
MongoDB test	127.0.0.1:27018	school_test	test_user
MongoDB simulated prod	127.0.0.1:27019	school_prod	prod_user

Summary: Four isolated databases are running locally, each reachable on its own port. Each stores a small *environment_info* record that names the real environment, which the Python program later reads back.

6. Task 4: MySQL with Values in the Code

In the first stage, *db_connect.py* connects to MySQL with every setting written directly in the source code. *user* and *password* supply the credentials, *host* and *port* identify the server and its MySQL service, and *database* selects the database. *mysql.connector.connect()* opens the connection. A cursor then runs the query *SELECT environment FROM environment_info* to read which environment the server belongs to, and both the cursor and the connection are closed to release resources.

```
import mysql.connector

connection = mysql.connector.connect(
    host="127.0.0.1",
    port=3307,
    user="mysql_student",
    password="********",
    database="school_mysql",
)

print("Connected to MySQL:", connection.database)
print("Server:", connection.server_host, connection.server_port)
cursor = connection.cursor()
cursor.execute("SELECT environment FROM environment_info")
print("Environment:", cursor.fetchone()[0])
cursor.close()
connection.close()
```

Result: The program printed *Connected to MySQL: school_mysql*, *Server: 127.0.0.1 3307* and *Environment: dev*. The database name, the endpoint and the value stored inside the database all agree with the expected MySQL configuration.

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ nano db_connect.py
(.venv) eman-fatima-7125@ubuntu:~/Databases$ python db_connect.py
Connected to MySQL: school_mysql
Server: 127.0.0.1 3307
Environment: dev
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 8: Successful MySQL connection with hard-coded values.

Summary: The connection works, but the settings are fixed inside the program, so changing the target would require editing the code.

7. Task 5: MongoDB with Values in the Code

The same file was then replaced with a MongoDB version. The username, password, endpoint and database name are kept in four separate variables, and the + operator joins them into a MongoDB URI of the form *mongodb://username:password@endpoint/database*. In this lab the database in the URI is also the authentication database. *MongoClient(uri)* creates the client and *client[database_name]* selects the database.

Creating a client alone does not contact the server, so *database.command("ping")* is used to send a real request and prove connectivity. *serverSelectionTimeoutMS=5000* limits the wait to five seconds so that a wrong endpoint fails quickly. The final line reads the *name* field from the *environment_info* collection.

```
from pymongo import MongoClient

username = "dev_user"
password = "********"
endpoint = "127.0.0.1:27017"
database_name = "school_dev"

uri = "mongodb://" + username + ":" + password + "@" + endpoint + "/" + database_name

client = MongoClient(uri, serverSelectionTimeoutMS=5000)
database = client[database_name]
database.command("ping")
print("Connected to MongoDB:", database.name)
print("Server:", client.address)
print("Environment:", database.environment_info.find_one()["name"])
client.close()
```

Result: The output shows database *school_dev*, server *('127.0.0.1', 27017)* and environment *dev*.

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ nano db_connect.py
(.venv) eman-fatima-7125@ubuntu:~/Databases$ python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 9: Successful MongoDB connection with hard-coded values.

Summary: Only the database driver changed; the command used to run the program stayed the same.

8. Task 6: MongoDB with os.getenv()

Hard-coded settings are inconvenient and unsafe, because secrets end up in source code and the program must be edited for every environment. In this stage the four values are read from environment variables with *os.getenv()*. Python inherits the variables exported in the shell, so the identical program can target dev, test or simulated prod simply by exporting different values. A short *if* statement stops the program with a clear message if any variable is missing. The URI construction, connection and output block remain unchanged.

```
import os
from pymongo import MongoClient

username = os.getenv("MONGO_USERNAME")
password = os.getenv("MONGO_PASSWORD")
endpoint = os.getenv("MONGO_ENDPOINT")
database_name = os.getenv("MONGO_DATABASE")

if not username or not password or not endpoint or not database_name:
    raise SystemExit("Set all four MongoDB environment variables first.")

# URI construction and MongoDB connection block: unchanged
...
```

8.1 Development values

The dev profile was exported and the program was run. The expected result is database *school_dev* on port 27017 with environment *dev*, and the screenshot shows exactly this.

```
export MONGO_USERNAME=dev_user
export MONGO_PASSWORD='********'
export MONGO_ENDPOINT=127.0.0.1:27017
export MONGO_DATABASE=school_dev
clear
python db_connect.py
```

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ export MONGO_USERNAME=dev_user
export MONGO_PASSWORD='DevDemoOnly123!'
export MONGO_ENDPOINT=127.0.0.1:27017
export MONGO_DATABASE=school_dev
python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 10: Environment-variable version using the dev profile.

8.2 Test values

The four variables were then re-exported for the test deployment, and the program was run again without any change to the code. For this profile the target should be database *school_test* on port 27018 with environment *test*.

```
export MONGO_USERNAME=test_user
export MONGO_PASSWORD='********'
export MONGO_ENDPOINT=127.0.0.1:27018
export MONGO_DATABASE=school_test
clear
python db_connect.py
```



Figure 11: Environment-variable version with the test profile exported.

8.3 Simulated production values

Finally the simulated production profile was exported. The expected target is database *school_prod* on port 27019 with environment *prod*.

```
export MONGO_USERNAME=prod_user
export MONGO_PASSWORD='********'
export MONGO_ENDPOINT=127.0.0.1:27019
export MONGO_DATABASE=school_prod
clear
python db_connect.py
```

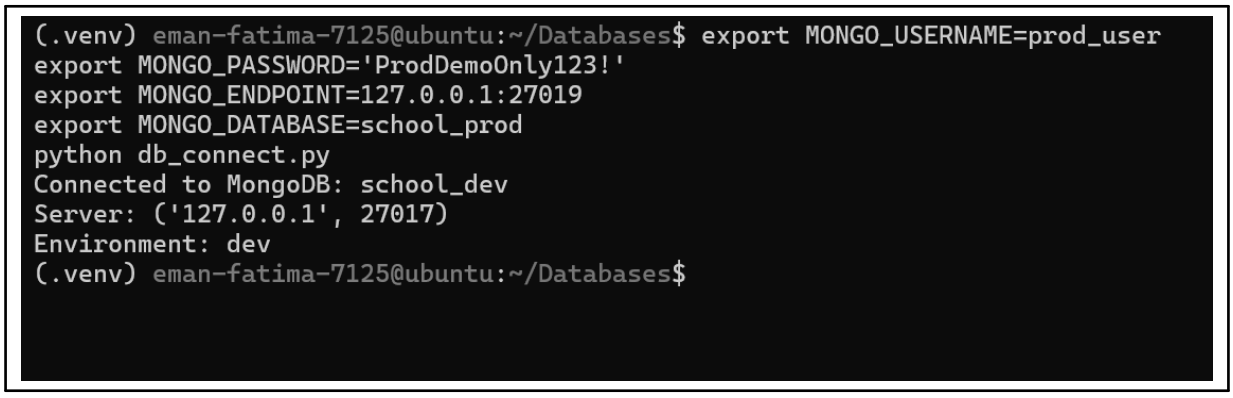


Figure 12: Environment-variable version with the simulated prod profile exported.

Profile	Expected database	Expected port	Expected environment
dev	school_dev	27017	dev
test	school_test	27018	test
prod	school_prod	27019	prod

Summary: One unchanged program can connect to three different databases because its settings come from the process environment rather than from the code.

9. Task 7: MongoDB with a Credentials File

Environment variables must be re-exported by hand each time. A credentials file groups all settings into named sections that can be selected with one variable. The instructor-supplied template *credentials.ini.example* was copied to a local file *credentials.ini*, which is excluded from Git so that secrets are never committed. The *-n* option prevents overwriting an existing file; the warning in the screenshot is only a notice that this option is being deprecated and does not indicate a failure. The following *ls* confirms that the file exists.

```
cp -n credentials.ini.example credentials.ini
ls credentials.ini
```

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ cp -n credentials.ini.example credentials.ini
ls credentials.ini
cp: warning: behavior of -n is non-portable and may change in future; use --update=none instead
credentials.ini
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 13: Creating credentials.ini from the example template.

The file contains three sections, *[dev]*, *[test]* and *[prod]*, each holding a username, password, endpoint and database name:

```
[dev]
username = dev_user
password = ********
endpoint = 127.0.0.1:27017
database = school_dev

[test]
username = test_user
password = ********
endpoint = 127.0.0.1:27018
database = school_test

[prod]
username = prod_user
password = ********
endpoint = 127.0.0.1:27019
database = school_prod
```

9.1 Restricting file permissions

Because an INI file stores secrets in plain text, access should be limited. *chmod 600* gives the owner read and write permission and removes all access for the group and other users. *stat -c '%a %n'* then prints the numeric mode, which is *600 credentials.ini* as required.

```
chmod 600 credentials.ini
stat -c '%a %n' credentials.ini
```

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ nano credentials.ini
(.venv) eman-fatima-7125@ubuntu:~/Databases$ chmod 600 credentials.ini
(.venv) eman-fatima-7125@ubuntu:~/Databases$ stat -c '%a %n' credentials.ini
600 credentials.ini
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 14: File mode of credentials.ini restricted to 600.

9.2 Reading settings with configparser

The program now uses Python's built-in *configparser* module. *config.read()* loads the file, and *config[environment]* selects a section whose name comes from *os.getenv("APP_ENV", "dev")*; if *APP_ENV* is not set, the *dev* section is used by default. *interpolation=None* makes special characters such as % be read literally. The connection and output block is exactly the same as in the previous stages.

```
import os
import configparser
from pymongo import MongoClient

config = configparser.ConfigParser(interpolation=None)
config.read("credentials.ini")
environment = os.getenv("APP_ENV", "dev")
username = config[environment]["username"]
password = config[environment]["password"]
endpoint = config[environment]["endpoint"]
database_name = config[environment]["database"]

# URI construction and MongoDB connection block: unchanged
...
```

9.3 Running the dev, test and prod sections

Selecting the dev section with *export APP_ENV=dev* produced database *school_dev*, server *('127.0.0.1', 27017)* and environment *dev*.

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ nano db_connect.py
(.venv) eman-fatima-7125@ubuntu:~/Databases$ export APP_ENV=dev
python db_connect.py
Connected to MongoDB: school_dev
Server: ('127.0.0.1', 27017)
Environment: dev
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 15: Credentials-file version using APP_ENV=dev.

Selecting the test section with *export APP_ENV=test* produced database *school_test*, server *('127.0.0.1', 27018)* and environment *test*.

```
(.venv) eman-fatima-7125@ubuntu:~/Databases$ export APP_ENV=test
python db_connect.py
Connected to MongoDB: school_test
Server: ('127.0.0.1', 27018)
Environment: test
(.venv) eman-fatima-7125@ubuntu:~/Databases$
```

Figure 16: Credentials-file version using APP_ENV=test.

Selecting the simulated production section with *export APP_ENV=prod* produced database *school_prod*, server *('127.0.0.1', 27019)* and environment *prod*.



Figure 17: Credentials-file version using APP_ENV=prod.

Summary: The same connection code now reads all of its settings from an external file, and only one variable, *APP_ENV*, is needed to switch between environments.

10. Exam Evaluation Questions

Question 1: A missing environment variable

Setup: With the valid prod profile exported, only *MONGO_DATABASE* is removed using *unset MONGO_DATABASE*, and the program is run again.

Prediction and explanation: The program stops with the message “Set all four MongoDB environment variables first.” The call *os.getenv("MONGO_DATABASE")* returns *None* because that variable no longer exists. The condition *not username or not password or not endpoint or not database_name* becomes true as soon as one value is empty or *None*, so *raise SystemExit* ends the program before any URI is built and before any connection to the database is attempted. Restoring *export MONGO_DATABASE=school_prod* makes all four values available again, and the program reconnects to *school_prod*.

Question 2: Which source does the file version read?

Setup: The prod username, password, endpoint and database variables from Task 6 remain exported, and only *APP_ENV* is removed with *unset APP_ENV*.

Prediction and explanation: The program connects to dev. The credentials-file version never calls *os.getenv()* for *MONGO_USERNAME*, *MONGO_PASSWORD*, *MONGO_ENDPOINT* or *MONGO_DATABASE*; the leftover prod variables are simply ignored. It reads only *APP_ENV* to choose a section of *credentials.ini*, and when *APP_ENV* is unset the default value *dev* is used. The source of the settings is therefore the file section and not the shell’s connection variables. Setting *APP_ENV=test* then connects to *school_test* on port 27018.

Question 3: A wrong password in the file

Setup: In the *[test]* section only, the password is temporarily changed to a wrong value, with *APP_ENV=test* exported before running the program.

Prediction and explanation: PyMongo raises an authentication error. The endpoint (127.0.0.1:27018) is correct, so the server is found and the network connection succeeds,

but the server also verifies the credentials. Locating the server and proving identity are two separate steps: a correct address only tells the client where to knock, while the password proves who is knocking. An incorrect password is rejected regardless of how correct the other settings are. Restoring the original test password makes the program connect to *school_test* again, after which *APP_ENV=prod* is restored. The credentials file is never committed to Git.

11. Overall Summary and Conclusion

Through four short stages, one Python file was evolved from a MySQL connection with hard-coded values into a flexible, configuration-driven MongoDB client. The stages show a clear progression in good practice:

- Values written in code are simple but inflexible and place secrets in the source.
- Environment variables via *os.getenv()* allow one program to target dev, test and prod.
- A credentials file with *configparser* groups the settings in sections selected by a single variable, and its permissions can be restricted with *chmod 600*.
- A real request (*ping*) and the stored *environment_info* value confirm which database actually answered.

Stage	Database	Expected database name	Port	Environment
Task 4: values in code	MySQL	school_mysql	3307	dev
Task 5: values in code	MongoDB	school_dev	27017	dev
Task 6: os.getenv, dev	MongoDB	school_dev	27017	dev
Task 6: os.getenv, test	MongoDB	school_test	27018	test
Task 6: os.getenv, prod	MongoDB	school_prod	27019	prod
Task 7: file, dev	MongoDB	school_dev	27017	dev
Task 7: file, test	MongoDB	school_test	27018	test
Task 7: file, prod	MongoDB	school_prod	27019	prod

After the evidence was captured, the databases were stopped with *docker compose down* from *~/Databases*; the option *-v* was not used so that no data volumes were deleted.

12. Submission Details

Item	Value
Student	Eman Fatima (2023-BCS-025), Section A
GitHub username	Eman-Fatima-7125
Databases fork URL	https://github.com/Eman-Fatima-7125/Databases
Submission directory	DevOps/Labs/Lab04 (README.md, Lab4_Solution.pdf, screenshots/)
Final db_connect.py	Credentials-file version

Official references

- MySQL Connector/Python: Connecting to MySQL
- MongoDB: Connect to MongoDB with PyMongo
- Python: os.getenv and configparser
- Python: virtual environments
- Docker: Install the Compose plugin manually; Linux post-installation steps
- Git: gitignore